ELLY Business Development Portal

Full Project Audit — Missing Features, Issues, Failures, Outstanding Work

---

Date: 2026-10-28

Repository: Business-Development-and-Management-OS_IHT (business-outreach)

Branch: main

Scope: Backend (FastAPI/SQLAlchemy/SQLite), Frontend (React/TypeScript/Vite), tests, CI, security, repo hygiene.

Method: Static code audit (grep/read), live backend wiring verification, targeted pytest runs, tsc type check, git history/gitignore checks. Read-only — no production data inspected or exposed.

Severity key

- **[CRITICAL]** — exploitable now / data loss risk / breaks core flow
- **[HIGH]** — real risk or gap, should be scheduled soon
- **[MEDIUM]** — real issue, not urgent
- **[LOW]** — hygiene / cleanup / cosmetic
- **[INFO]** — confirmed fine, noted for completeness

1. Executive Summary

The project is in materially better shape than the last written plan suggested. A prior plan (Phases 0-9, 'Port remaining IHT subsystems') assumed scheduler, notifications, signals, knowledge base, webhooks, and two-way Sheets sync were unbuilt. This audit found **all of them are now implemented and wired end-to-end** — models, services, routers, and (where relevant) APScheduler jobs and frontend UI all connect correctly. That plan is stale and should be retired or rewritten.

No critical security vulnerabilities were found. Secrets are gitignored and never committed to history. The inbound webhook HMAC check, JWT secret enforcement, and server-side pre-send gate are all correctly implemented.

The most consequential open issues are: (1) no database migration tool (Alembic) — schema changes rely on ad-hoc ALTER TABLE code, risky as the schema grows; (2) the full backend test suite has a ~50% error rate when run end-to-end, almost certainly from live external API calls without mocking/isolation — this masks real regressions; (3) the nightly backup job has not produced a backup in 3 weeks despite being correctly scheduled, implying the server isn't kept running continuously; (4) several newer subsystems (notifications, webhooks, knowledge base, backups, signals) have zero dedicated test coverage even though they are live and wired.

Update: since the original audit, five dead/unused folders have been removed from disk entirely — `OpenOutreach/` (~690MB), `Scrapegraph-ai/` (~25MB), `whatsapp-extension/`, `linkedin-extension/` (both superseded by elly-outreach-extension/), and `Finance-Management-OS_IHT/` (~58MB, after confirming — file by file — that business-outreach already has an equivalent or superior implementation of every non-excluded feature it contained). All five were gitignored with zero tracked files, so the deletions are not reflected in git history and cannot be undone from git. `leads-business/` is the only large reference folder still present.

2. Backend — Architecture, Routers, Services, Models

2.1 Phase 0-9 subsystem plan — status correction

The plan file `serialized-greeting-sunrise.md` describing a phased port of IHT subsystems is **out of date**. Verified wiring, not just file existence:

- **[INFO]** Scheduler: 5 APScheduler jobs registered in `main.py:380-384` on app lifespan — nightly backup (02:30), task due-sweep (30min), reply sync (5min), signal scan (daily 06:00), sheets pull (15min). Each job wrapped so one failure cannot kill the loop.
- **[INFO]** Notifications: `notification_service.notify()` called from 5 real call sites (integrations, campaign_service, lead_tracker_service, signal_service, task_sweep_service) — not orphaned.
- **[INFO]** Knowledge Base: wired into Copilot as a live tool (`kb_search`, tools.py:1210-1211).
- **[INFO]** Webhooks: both inbound (HMAC-verified) and outbound (`dispatch_outbound()`, HMAC-signed, webhook_service.py:56-85) exist, plus subscription CRUD.
- **[INFO]** Google Sheets: real two-way sync — `pull_updates()` (line 364) and scheduled `pull_updates_job()` (line 452) exist; not push-only as previously assumed.
- **[MEDIUM]** Backups: service and nightly job are correctly wired, but `backend/backups/` contains only one file, dated ~Oct 6-8 — about three weeks stale as of this audit (2026-10-28). The job only runs while the server process is alive; if the dev/prod server is restarted or not kept running overnight, no backup happens. No rotation has ever been exercised in practice.
- Action: retire or rewrite the stale IHT-port plan file — most of its phases are done.

2.2 Code quality / dead code

- **[LOW]** No real TODO/FIXME/HACK stubs found in `app/`. Only hit was a defensive NotImplementedError catch in `research/stagehand_fallback.py:40` (intentional fallback).
- **[INFO]** No orphaned/dead model classes — TaskComment, Signal, Notification, WebhookSubscription, KBDocument all referenced outside their own file.

- **[LOW]** `backend/_pw_funding_check.py` — ad-hoc Playwright debug script, not part of the app. Safe to delete.
- **[MEDIUM]** `backend/reset_database.py` — legitimate destructive utility (wipes test data, keeps users + showcase projects). Should require explicit confirmation before ever being run against a real database; no safeguard currently prevents accidental execution against `bd_portal.db`.

2.3 Data layer / migrations

- **[HIGH]** No Alembic (or any migration tool) anywhere in application code — confirmed via repo-wide search (only hits are in `.venv` dependencies and unrelated vendor folders). The sole schema-evolution mechanism is `migrate_db_columns()`, an ad-hoc ALTER TABLE function in `database.py`. This has no rollback, no version history, and no safety net against drift between environments. **Recommended before the schema grows further.**
- **[INFO]** SQLite concurrency hardening (WAL mode, busy_timeout=30000, NullPool) already fixed earlier this session — confirmed still in place and correct.

2.4 main.py / configuration

- **[INFO]** Middleware order is sound: SecurityHeadersMiddleware, GmailRateLimitMiddleware, RateLimitMiddleware(120/min), CORSMiddleware.
- **[INFO]** `DEBUG` defaults to false; startup hard-fails if critical secrets are unresolved under DEBUG=false — a good guardrail already in place.
- **[MEDIUM]** `CORS_ORIGINS` defaults to localhost only (5173/3000) — fine for current single-team use, but must be set to the real production origin before any external deployment.

3. Security & Secrets

No secret values were read or printed as part of this audit — only file existence, gitignore status, and structural facts.

3.1 Findings

- **[HIGH]** JWT access token stored in plain `localStorage`, not an httpOnly cookie (`frontend/src/context/AuthContext.tsx:22,39,48,73,74`). Any XSS on any page can exfiltrate the token. Common SPA trade-off, but worth hardening before wider exposure.
- **[HIGH]** Rate limiter state is an in-memory Python dict (`backend/app/core/middleware.py:17`). Resets on every restart and is per-worker-process — silently stops limiting the moment the app runs with more than one uvicorn/gunicorn worker.
- **[HIGH]** `backend/.env` has duplicate `JWT_SECRET_KEY` and duplicate `TOKEN_ENCRYPTION_KEY` lines (confirmed structurally, values not read). The later line wins via python-dotenv; not a live bug today, but a real risk of someone editing the wrong (dead) copy later.
- **[MEDIUM]** `backend/.env` is missing several variable names present in `.env.example` (GEMINI_API_KEY, GROQ_API_KEY, NVIDIA_NIM_API_KEY, SERPER_API_KEY, TAVILY_API_KEY, SCRAPEGRAPH_ENABLED, GMAPS_SCRAPER_* vars) — name-only comparison. Needs a live check: either these load from elsewhere/have safe defaults, or the features they gate are silently disabled.
- **[MEDIUM]** `CORS_ORIGINS` default is dev-only (see 2.4) — repeated here as a security-relevant item, not just a config nit.

3.2 Confirmed sound (no action needed)

- **[INFO]** JWT secret has no hardcoded production fallback — auto-generated throwaway only under DEBUG=true; startup hard-fails in production if the secret is unset or under 32 characters (`config.py:10-21, 130-131`). Well designed.
- **[INFO]** Inbound webhook HMAC-SHA256 verification against `WEBHOOK_INBOUND_SECRET` is actually enforced (`routers/integrations.py:87-94`), not a dead config value.
- **[INFO]** `presend_gate.py` is pure server-side logic, invoked from both CampaignService and GmailService send paths — not client-bypassable.
- **[INFO]** No hardcoded API keys, passwords, or tokens found anywhere in `.py`/`.ts`/`.tsx` files — all secrets route through `.env`.
- **[INFO]** `bd_portal.db`, `test_bd.db`, `.env`, and all three credential JSON files (gmail_token.json, google_credentials.json, google_gmail_credentials.json) are gitignored **and** confirmed never committed in git history (checked via `git log --all --full-history` on each).
- **[INFO]** `whatsapp-extension/` and `linkedin-extension/` contain only extension code, no stored session cookies on disk; both gitignored.

4. Frontend

- **[LOW]** One TypeScript error: `src/pages/leads/LeadTrackerPage.tsx:114` — `TS2503: Cannot find namespace NodeJS`. Likely a `useRef<NodeJS.Timeout>` typing; fix by using `ReturnType<typeof setTimeout>` instead.
- **[INFO]** All 65 page components under `src/pages/**` are routed in App.tsx — no orphaned pages, no route pointing to a missing component.
- **[INFO]** No "Coming Soon"/"Not implemented" stub markers found anywhere in the UI.
- **[LOW]** Near-clean of debug leftovers — only one file (CalendarPage.tsx) matched for TODO/FIXME/console.log/debugger combined.
- **[MEDIUM]** No frontend service or UI exists to manage webhook subscriptions, even though the backend fully supports inbound + outbound webhooks (models, service, HMAC signing). The feature is backend-only and invisible to users.
- **[INFO]** notifications.ts, signals.ts, tasks.ts, knowledgeBase.ts all exist and call real, matching backend endpoints — confirms those subsystems are wired frontend-to-backend, not just backend stubs.

5. Tests & CI

5.1 Suite health

- **[HIGH]** No CI pipeline exists — `.github/workflows` is absent from this repo (hits only appear in unrelated nested vendor folders). Tests only ever run manually, if at all.
- **[HIGH]** Full suite (603 tests, 41 files) was run and showed a systemic ~50% error rate (~113 passed / ~111 errored / ~1 failed, from a representative ~37% partial run before it was stopped as too slow to finish in this audit's time budget). The error pattern was uniform throughout — not random flakiness, not a hang.
- **[HIGH]** Most likely cause of the above: tests making live external API calls (Tavily, Serper, ScrapeGraph, Gmail, Groq) with no mocking layer and no per-test network isolation, so failures cascade once rate limits or network/API-key state is hit partway through a long run — the same failure shape as the database-session-poisoning bug already fixed once this session in lead-import. **Needs a dedicated debugging pass** — this is not confirmed root cause, just the leading hypothesis from evidence gathered.
- **[INFO]** Targeted isolated run of the two uncommitted Harvey P0-P2 test files — `test_p0_send_gate_and_email_accuracy.py` + `test_p1_site_profiler_and_provenance.py` — passed **61/61** in 16.6s with zero failures. The new send-gate/email-accuracy/site-profiler work is solid on its own, in isolation. This is a real signal that the 50% suite-wide error rate is environmental (see above), not a defect in the newest work.
- **[MEDIUM]** No coverage tool configured (`pytest-cov`/`coverage` absent from requirements.txt) — test coverage is never measured.
- **[LOW]** Zero `@pytest.mark.skip` / `@pytest.mark.xfail` markers anywhere — no tests are being silently skipped, which is good, but also means the ~50% error rate above is not being swept under a skip marker; it is a live, unaddressed problem.

5.2 Coverage gaps for newer subsystems

The following live, fully-wired subsystems (see Section 2.1) have zero dedicated test coverage: **notifications**, **webhooks**, **knowledge base**, **backup_service**. **Scheduler** appears only incidentally in one broader test file. **Signals** have effectively no real coverage (only incidental string matches, no actual test of signal_service or the Signal model).

6. Repo Hygiene

6.1 Nested reference/stray folders at repo root

Folder	Size	Git status	Note
OpenOutreach/	~690 MB	REMOVED	Deleted from disk — was reference-only, unused
Finance-Management-OS_IHT/	~58 MB	REMOVED	Deleted from disk — confirmed no net-new features vs business-outreach
Scrapegraph-ai/	~25 MB	REMOVED	Deleted from disk — was reference-only, unused
whatsapp-extension/	36 KB	REMOVED	Deleted from disk — superseded by elly-outreach-extension/
linkedin-extension/	44 KB	REMOVED	Deleted from disk — superseded by elly-outreach-extension/
leads-business/	~5.7 MB	Ignored, 0 tracked	Unrelated local project nested in working dir — still present
elly-outreach-extension/	60 KB	7 files TRACKED	Live replacement — correctly in-repo

- **[INFO]** `OpenOutreach/`, `Scrapegraph-ai/`, `whatsapp-extension/`, `linkedin-extension/`, and `Finance-Management-OS_IHT/` — all flagged in the original audit — have since been deleted from disk entirely. None were git-tracked, so no commit was needed.
- **[LOW]** `leads-business/` remains as the only large reference-project folder still present — correctly gitignored, but a mild onboarding/disk-usage smell for new contributors.

6.2 Loose files at repo root

- **[LOW]** `bd_portal.db` and `test_bd.db` at repo root (not `backend/`) are stale orphaned leftovers from a process once started from the repo root — not git-tracked, not referenced by any code path (DATABASE_URL is a relative path that resolves against `backend/` in normal operation). Safe to delete.
- **[MEDIUM]** `elly-architecture.html`, `elly-architecture.visual-check.json`, `architecture.json`, and `phase23_live_validation.xlsx` are all **git-tracked** generated/exported artifacts, not source. They bloat diffs every time they are regenerated. Recommend moving to an ignored `artifacts/` path or gitignoring outright.
- **[INFO]** Upload directories (`backend/uploads/` — 831 files/49MB, `backend/private_uploads/`, root `uploads/`) are all correctly gitignored and contain real runtime data, as expected.
- **[INFO]** `.git` directory itself is a healthy 5.0MB — no binary/DB bloat in history.

7. Other Outstanding Items (carried over, not newly discovered)

Items already known from recent work this cycle, included here for a complete picture:

- **[MEDIUM]** Agency-agents vetting decision — a curated set of ~20 sales/marketing/finance/support/project-management agent definitions proposed for global install into `~/.claude/agents/` — still awaiting a go/no-go decision.
- **[INFO]** An in-progress design for a new autonomous, confirmation-gated prospect-discovery pipeline in Copilot (replacing the current ungated auto-save path in `discover_qualified`) was being brainstormed and is paused mid-approval, not abandoned.
- **[LOW]** A second, smaller feature ask — per-message edit/update/remove/comment actions in the Copilot chat UI — has been scoped as a separate follow-on but not yet brainstormed in detail.

8. Prioritized Action List

#	Action	Severity	Effort
1	Debug the ~50% systemic pytest error rate (likely live external API calls, no mocking)	HIGH	Medium
2	Add Alembic migrations before further schema changes	HIGH	Medium
3	Move rate-limiter state out of in-process memory (or document single-worker constraint)	HIGH	Small
4	Clean up duplicate JWT_SECRET_KEY / TOKEN_ENCRYPTION_KEY lines in backend/.env	HIGH	Trivial
5	Set up a CI workflow (.github/workflows) to run tests on push/PR	HIGH	Medium
6	Confirm backup job is actually running on a continuously-up server; investigate 3-week gap	MEDIUM	Small
7	Add test coverage for notifications, webhooks, knowledge base, backups, signals	MEDIUM	Medium
8	Un-track generated artifacts (elly-architecture.html, architecture.json, xlsx) from git	MEDIUM	Trivial
9	Delete stale root-level bd_portal.db / test_bd.db leftovers	LOW	Trivial
10	Fix NodeJS.Timeout TS error in LeadTrackerPage.tsx	LOW	Trivial
11	Build webhook subscription management UI (currently backend-only)	MEDIUM	Medium
12	Retire/rewrite the stale IHT subsystem-port plan file	LOW	Trivial
13	Decide on JWT storage hardening (httpOnly cookie vs localStorage) before wider exposure	HIGH	Medium
14	Resolve agency-agents global install decision	MEDIUM	Trivial

End of report. This audit was generated via static code analysis, targeted test execution, and git/gitignore inspection — no production data was read or exposed in the process.